

Detecting Malicious Chrome Extensions: Static Analysis Heuristics and Scoring

Introduction

Modern malicious Chrome extensions pose significant security risks by abusing browser privileges to steal data, hijack sessions, and execute unauthorized code. Attackers increasingly target Manifest V2 and V3 extensions for **token theft, session hijacking, credential or OTP interception, fake AI tool impersonation, and even supply-chain takeovers**. Building a static analysis tool to scan CRX→ZIP extension packages can help detect such threats before they execute. This report surveys **implementable static-analysis heuristics and a scoring model** for Chrome extensions, focusing on manifest and code scanning (with IOC extraction and correlation rules) to flag malicious behavior. We also outline top behavioral patterns to detect, strategies for identifying obfuscation and remote code-fetching, risky permission combinations, anti-analysis tricks, and techniques to minimize false positives. Finally, we provide a prioritized implementation backlog and a suggested JSON output schema for detection results.

Key Malicious Behaviors in Extensions

Understanding common malicious behaviors is critical to inform our detection rules. Malicious extensions typically aim to:

- **Steal authentication tokens & cookies (session hijacking)** – e.g. reading session cookies or local storage to hijack login sessions ¹ ². This allows attackers to impersonate users on sites like Facebook, Google, or enterprise apps without needing passwords.
- **Capture credentials and OTPs** – e.g. keylogging login forms, intercepting HTTP auth prompts, or reading one-time passcodes from webpages or emails. Some extensions inject scripts into login pages or use webRequest listeners to grab credentials in transit ³ ⁴.
- **Exfiltrate sensitive data (API keys, chats)** – e.g. scraping content from web pages (like ChatGPT conversations or emails) and sending it to attacker servers ⁵ ⁶. “Fake AI” extensions often impersonate AI tools to trick users and then silently capture chatbot conversations, browser history, or form inputs.
- **Perform session proxies and MitM** – e.g. malicious VPN/proxy extensions that route traffic through attacker-controlled servers. They may intercept all web traffic or specific high-value domains to capture passwords, credit card numbers, and tokens in real time ⁷ ⁴.
- **Inject ads or phishing content** – e.g. inserting unwanted ads or fake login forms by modifying pages. Some extensions overlay phishing prompts or redirect pages to malicious sites.
- **Abuse supply-chain trust** – e.g. an extension with a large user base gets compromised via a developer account takeover, and a malicious update is pushed ⁸ ⁹. The new version often includes obfuscated code that fetches instructions from a remote C2 (Command-and-Control) server, enabling data theft or further compromise.

These behaviors often **evade naive detection** because extensions hide malicious code behind obfuscation, conditional triggers, or masquerade as legitimate functionality. A robust static analyzer must therefore combine multiple heuristics, scanning both the **manifest (for dangerous permissions/configurations)** and **code files (for suspicious patterns and IOCs)**.

Static Analysis Heuristics for Malicious Extension Detection

In a static CRX analyzer, we will implement a set of heuristic rules that score an extension's risk. Each rule examines either the manifest or the JavaScript/HTML source for red flags. Below we detail the top behavioral patterns and indicators:

1. Dangerous Permission Combinations (Manifest Analysis)

Chrome extension manifests declare what sites and browser features the extension can access. Malicious extensions often request **excessive or incongruous permissions** that enable broad surveillance or tampering. Key flags include:

- **All-sites access:** `host_permissions` or content script matches set to `<all_urls>` (all HTTP/HTTPS sites) or overly broad patterns. Full access to all sites means the extension can inject scripts or read data on any webpage ². If an extension claims to do a narrow task (e.g. "theme" or UI tweak) but asks for all-site access, it's highly suspicious ¹⁰.
- **Web Request Interception:** Permissions like `"webRequest"` (especially coupled with `"webRequestBlocking"` or the MV3 `"declarativeNetRequest"` APIs). These allow the extension to intercept and modify network traffic or HTTP headers. In malicious hands, this enables **man-in-the-browser** attacks – e.g. stripping security headers, sniffing API calls, or injecting authentication data ¹¹. A known malicious extension abused `declarativeNetRequest` to remove Content Security Policy protections and inject scripts into pages ¹².
- **Cookies and Sessions:** The `"cookies"` permission combined with broad host access lets an extension read any site's cookies. This can be used to steal session tokens (e.g. an extension that grabs your Facebook or cloud service cookies for hijack) ¹. Likewise, `"tabs"` permission with `<all_urls>` access can expose session details (by allowing script injection or reading page content).
- **Proxy/Network Settings:** Permissions like `"proxy"` or `"declarativeNetRequestWithHostAccess"` signal the extension can **reroute traffic**. If paired with broad host permissions, it could funnel user traffic through an attacker's proxy for surveillance ⁷ ⁴. For example, a recent malware extension registered a proxy auto-config (PAC) script to siphon traffic from 170+ domains (cloud services, social media, dev platforms) to its servers ¹³ ¹⁴.
- **Scripting and Execution:** In MV3, `"scripting"` permission (or in MV2, use of `chrome.tabs.executeScript`) allows injecting arbitrary scripts into pages. If an extension can inject code into every website, it might be used to **keylog or modify pages** to collect credentials ¹⁵. Check if `"content_scripts"` in the manifest are set to run on all frames and at `document_start` on all sites – this combination means the extension's content script executes **on every page load** before the page scripts, ideal for snooping or tampering.
- **Powerful but Rare Permissions:** Flags like `"debugger"` (which lets an extension attach to and instrument web pages), `"nativeMessaging"` (to launch native apps on the system), `"downloads"` (could read or save files), `"history"` (read browsing history), or `"management"`

(to control other extensions). These are not inherently malicious but are unusual for most benign extensions. If present without a clear justification, they raise risk. For instance, an extension using "management" might attempt to disable security extensions or hide itself.

Heuristic: Develop a **permission risk score**. Assign higher weights to critical permissions, especially in **dangerous combinations**. For example, an extension with `<all_urls>` + `cookies` + `declarativeNetRequest` would score very high (full content and network interception) ² ¹¹. A simple ad-blocker might need `<all_urls>` and DNR, but it should not be accessing cookies or making external script calls. Compare the declared permissions against the extension's described purpose (if available) to flag mismatches (e.g., a "PDF converter" extension asking for `webRequest` or cookie access is suspect). This forms the baseline of the static analysis scoring, as **over-privileged extensions account for a large share of malicious cases** ¹⁶ ¹⁷.

2. Suspicious API Usage Patterns in Code

Beyond permissions, we scan the extension's JavaScript for code patterns commonly used in malicious behavior:

- **Credential or Token Access:** Look for code that programmatically reads sensitive data:
 - Calls to `chrome.cookies.get()` / `getAll()` or usage of `document.cookie` in content scripts ¹.
 - Accessing web storage like `localStorage` or `sessionStorage` on web pages (often via content scripts) for items named like "token", "session", "auth", etc.
 - Functions hooking into page forms: e.g., adding event listeners for "submit" on forms or "input" on password fields, or scanning the DOM for `<input type="password">` fields. Legitimate extensions rarely need to handle raw password inputs.
 - Use of Chrome's `chrome.webRequest.onBeforeRequest` or `onHeadersReceived` listeners to intercept login requests, or `onBeforeSendHeaders` to capture Authorization headers. For example, malicious extensions have hooked `onAuthRequired` events to supply attacker-controlled credentials and hijack HTTP auth dialogs ³. If code registers an `onAuthRequired` listener (especially with `asyncBlocking`), that's a strong indicator of credential interception in progress ¹⁸.
- **Keystroke Logging and DOM Scraping:** Code that injects keyloggers or scrapes page content:
 - Use of low-level DOM events like `document.addEventListener('keydown', ...)` across all sites, or scripts that iterate over all input fields to read values.
 - If the extension's scripts query for specific DOM elements that contain sensitive info. For instance, a malicious AI extension might search for known ChatGPT conversation container elements on the page to extract chat text ⁶. Similarly, scanning for keywords like "OTP" or patterns of 6-digit codes in pages (indicative of 2FA codes) could indicate OTP theft attempts.
 - Direct DOM extraction: e.g., `document.body.innerText` or specific selectors on sites like Gmail, Facebook, etc., followed by sending that data out.
- **Network Exfiltration:** Patterns where data is sent to external servers:
 - Any usage of `fetch()`, `XMLHttpRequest`, `navigator.sendBeacon`, or even creating an `Image()` / `<img>` tag with an external URL (a common trick for stealthy GET requests) should be inspected. Analyze the target URLs or endpoints used – unknown domains or IP addresses (especially hardcoded) are IOCs to extract. For example, a supply-chain attack extension contacted domains like `graphqlnetwork[.]pro` and others to fetch instructions ⁹ ¹. Another case had

an extension sending chat data periodically to domains like `chatsaigpt[.]com` and `deepaichats[.]com` ⁶ .

- Check if the code packages data from the browser to send out: e.g., collecting cookies, tokens, or user inputs and then posting via `fetch`. The presence of keywords like `"Authorization"`, `"Bearer"`, or function names suggesting encryption (for payload) could indicate exfiltration of auth tokens or keys.
- Heartbeat or periodic beacons: Malicious extensions sometimes send regular pings with stolen data. For instance, an extension might use `setInterval` or alarms to send a heartbeat containing user credentials or session info every few minutes ⁴ . Statistically, benign extensions rarely transmit such sensitive info on an interval.
- **Remote Payload Delivery:** Many malicious extensions employ a **"two-stage"** approach: minimal malicious code in the extension, which then fetches the real payload or config from a remote server (to evade static store checks or to update functionality flexibly ¹⁹). To catch this:
 - Look for any network request soon after installation or on startup (`chrome.runtime.onInstalled` or `onStartup` handlers making fetch calls). If an extension downloads JavaScript or JSON from an external site, that's a red flag. For example, one malicious extension fetched a JSON "configuration" from a C2 which listed targeted domains and API endpoints to steal (like OpenAI and Facebook API URLs) ¹ ²⁰ . The static analyzer should flag code that reads config from a URL and then uses it to decide what data to collect.
 - Detect **dynamic code execution**: code that takes a remote response and executes it. This could be explicit (calling `eval` on response text, or using `new Function(...)` on fetched data) or obfuscated (e.g., storing fetched code in `chrome.storage` then later injecting it, or setting `innerHTML` / `outerHTML` of a page element with a `<script>` tag string).
 - Specific patterns: In MV2, presence of `eval()` or `Function()` is itself suspicious (especially if the manifest had to allow `'unsafe-eval'` in the content security policy). In MV3, `eval` is disallowed by default, but attackers use creative tricks: for instance, writing a script text into a DOM element's event handler attribute and triggering the event to execute it ²¹ . One advanced extension stored attacker code in a JSON config (`checklist.info` field) and used a content script to assign it to `document.documentElement.onreset`, then dispatched a `reset` event – causing the browser to execute that string as page context code ²² . Our static heuristic should catch weird patterns like setting attributes beginning with "on" (onreset, onload, etc.) with long suspicious strings, or use of `createElement('script')` with a remote URL.
 - The presence of `chrome.scripting.executeScript` (MV3) or `chrome.tabs.executeScript` (MV2) that pulls code from extension storage or an external source and injects it into pages is a strong signal. Also, adding `chrome.declarativeNetRequest.updateDynamicRules` with data from a remote response should be flagged – e.g. one malware used remote JSON to dynamically add blocking rules and also smuggled a code string in that JSON to execute on pages ²³ ²⁴ .
- **Abuse of Chrome APIs for malicious ends:** Certain API usage makes sense only for malicious scenarios:
 - Frequent use of `chrome.storage` to stash large blobs of data (especially binary blobs or long base64 strings) can indicate storing a payload or collected data before exfiltration.

- Using `chrome.identity` or OAuth tokens and then sending them out (some malicious extensions have abused OAuth flows to grab access tokens).
- Using `chrome.debugger` API to inspect network requests of a tab (rare, but possible).
- If the extension has a `devtools_page` (meaning it can run in Chrome DevTools), see if it uses `chrome.devtools.network` to tap into requests when DevTools is open – unlikely for malware targeting general users, but worth noting.

Heuristic: Maintain a **set of code pattern signatures** for malicious behaviors. This could be implemented via regex/AST patterns or lightweight static taint analysis. For each script file: - Flag any occurrences of dangerous API calls (cookies, webRequest, eval, execScript, etc.) and see if they are used in suspicious ways (e.g., fetching data then eval). - Extract all URL/domain strings and classify them (internal vs external, known vs random). External domains that don't match the extension's official website or are known bad are output as IOCs for further checking. - If a script contains an unusually long array of domain names or keywords (target list), that's suspicious. Malicious extensions often have a hardcoded list of high-value targets (e.g., dozens of crypto wallet domains, banking sites, or in one case 170+ service domains for proxying ²⁵). A benign extension rarely includes 100+ unrelated domain strings in its code. - Combine these signals to rate the code's malicious likelihood. For example, *"Content script reads document.cookie and sends to https://<random>.ru"* would trigger multiple rules (cookie access + external send) yielding a high score.

3. Obfuscation and Packing Detection

Malicious extensions almost always obscure their code to hinder manual review. Static analysis can use heuristics to detect obfuscated or encrypted code blocks:

- **Minified or Unreadable Code:** Calculate metrics like the average length of identifier names, or percentage of printable characters. If variable/function names are one or two letters long throughout a file, or the code has extremely long lines with no whitespace, it's likely machine-generated or minified. Many benign extensions use some minification, but **extreme minification combined with other factors (like presence of eval or suspicious strings)** should increase suspicion.
- **High Entropy Strings:** Scan for large base64, hex, or similarly encoded strings. Attackers may encode URLs or payloads to hide them. If we find long gibberish strings or evidence of chunks of data being decoded (look for usage of `atob()`, `cryptoJS.decrypt`, `String.fromCharCode` loops, etc.), that suggests the extension is unpacking something at runtime. For instance, a known malicious extension's payload was "heavily obfuscated... with most strings and property names encrypted" by a JS obfuscator ²⁶. Our tool should flag such encryption routines.
- **Known Obfuscator Signatures:** Many JS obfuscation tools leave telltale patterns or metadata. The Google Web Store scanners used regexes to catch code watermarks from common packers (e.g., `eval(function(p,a,c,k,e,d)...` from the classic Dean Edwards packer, or specific comments like `/* Powered by Obfuscator.io */`). We can implement detectors for known obfuscation frameworks or boilerplate initialization code ²⁷. For example, check for strings like `"\\x"` (hex escapes), suspicious use of `![]+[]` or `!![]` (JSFuck style), or self-defining functions that decode long strings.
- **Differences between prettified and original code:** A technique is to run a beautifier on the code and compare structure. If the original is one-liner or highly compressed versus a nicely formatted version, minification is confirmed ²⁷. While minification alone is not malicious, combine it with

context (like if a small extension is entirely one huge minified script with no open-source libraries identified).

- **Multiple Layers:** Some malicious extensions include a benign-looking small script that dynamically loads or reconstructs a bigger script (even splitting it across many eval calls). If we see any code that builds functions from pieces or uses techniques like constructing code via array of char codes, that's a strong indicator of obfuscation.
- **Unused or Dead Code:** Another heuristic – if the extension package includes large libraries or code that aren't actually used for the stated functionality, it could be camouflage or padding. For example, bundling an entire `jquery.js` just to hide a few lines of malicious code appended to it (as happened in one case) ³. We can detect if known libraries are modified (hash known library files and compare or look for extra code at the end of them).

Heuristic: Assign an **obfuscation score** to each script. If an extension has a high obfuscation score *and* requests sensitive permissions or hits any network/credential rules, then elevate its overall risk. Conversely, purely obfuscated but low-permission extensions might be PUPs (potentially unwanted, like ad injectors) but not outright credential thieves. The tool could also list which files are obfuscated to aid analysts. (Note: some benign extensions use minification or packers for performance – to reduce false positives, consider whitelisting known library scripts by checksum or filename, so we don't flag e.g. `react.min.js` as malicious.)

4. Remote Configuration & Code Fetch Detection

As noted, many attacks rely on pulling config or code from the web at runtime. Static analysis can't fetch those remote payloads (and shouldn't, to avoid contacting malicious servers during analysis), but it **can identify the intent to do so**:

- **Outbound URL Indicators:** Extract all URL strings in the code. If you see references to non-Chrome domains (especially those not obviously related to the extension's functionality or publisher), list them as IOCs. Then apply rules:
- If the extension contacts domains that are known bad or appear randomly generated/low reputation (e.g. unusual TLDs like `.pro`, `.info`, `.xyz`, IP literals), that's a high indicator. The supply-chain attack in late 2024 used dozens of new domains on TLDs like `.pro`, `.live`, `.info` as C2 for config files ⁹ ²⁸. Such domains can be cross-checked with threat intel sources.
- Check if the same domain appears in multiple extension scans – might indicate a campaign. Our system could maintain a database of malicious domains from prior detections and flag reuse.
- **Config Parsing Logic:** Identify code that handles JSON or other config data from network. E.g., look for `if (response.something)` or usage of JSON keys that hint at instructions. In one case, the extension expected a JSON with keys like `"graphqlnetworka" ... "graphqlnetworkk"`, which corresponded to different target URL patterns ²⁹. If the code contains specific key names or processes dynamic rules (like `updateDynamicRules` based on fetched data), we suspect remote config is guiding malicious behavior ²³.
- **Dynamic Injection Mechanisms:** As described before, detect indirect eval patterns. Some extensions fetch harmless-looking data then use clever mechanisms to execute hidden code within it. For example, Palantir's analysis revealed a content script storing fetched info in a DOM event attribute to execute code globally ²². Our tool should flag the combination of network fetch + setting `on*` attributes or injecting script tags.

- **Use of `web_accessible_resources`**: Check the manifest's `web_accessible_resources`. If an extension makes internal files accessible to web pages, it could allow a website to request those files and maybe exploit something. Notably, if an extension's own script is listed as web-accessible and the extension fetches remote data into that script, a website could potentially include that as `<script src="chrome-extension://id/evil.js">`. This is more a vulnerability than a malware tactic, but worth noting. Also, if they expose sensitive resources (like an internal config) to any webpage, that's unusual.

Heuristic: If an extension's code both **fetches from** and **executes** or uses external data in privileged context, mark it with a strong warning. The presence of any hardcoded C2 domains or IPs is an IOC to report (with context which file/line it came from). Our output should highlight these domains so a security analyst can pivot on them (e.g., see if they're known malicious or part of a larger campaign). Even if we can't know what the remote code does, the mere capability is often enough to label the extension malicious – for instance, Google's extension vetting treats fetching remote code as a serious violation ³⁰ ³¹.

5. Anti-Analysis and Evasion Tricks

Advanced malicious extensions try to avoid detection by automated scanners and researchers. Static analysis can catch some of these tricks:

- **Delayed or Conditional Execution:** If the code seems to **time-delay** its payload, that's a warning sign. E.g., using `chrome.alarms` or long `setTimeout` to wait hours or days before activating malicious functions. One documented case set a cookie on install and only after ~2 weeks of the user having the extension did it deliver the real payload ³² ³³. While static analysis can't know the time, it can detect use of `chrome.storage` or cookies to keep state and check for age. If you see code checking the current date/time or counting how long since install, it's likely trying to **defeat short-lived analysis sandboxes**.
- **Environment Checks:** Some malware might check for clues of being in an analysis environment. For example, looking at `navigator.webdriver` (headless browser indicator), or the presence of devtools. If static code reads properties like `window.navigator` or unusual browser specifics without a functional reason, note it. Another trick is to only act if certain user profile data is present (e.g., only run if user is logged into specific sites). That logic might appear as checks for particular cookies or localStorage keys (like only steal tokens if `localStorage['userEmail']` exists, etc.).
- **Innocuous Mode vs Malicious Mode:** Malicious extensions often have dual behavior. Immediately after installation or under certain conditions, they act normal (or do nothing) to pass Web Store review. Later, they might download an update or receive an activation signal. For static analysis, a telltale sign is **unused permissions or code paths** that don't align with the advertised functionality. If you see code that *could* steal data but it's gated behind some condition (like `if(config.active)` or `if(day > X)`), it's likely dormant malicious logic. Mark code branches that appear to do sensitive actions but are off by default.
- **Hiding Malicious Code in Legit Libraries:** Attackers sometimes insert their code into a minified third-party library (so the file looks like a known library at first glance). Our tool can detect known library files via hash or heuristic, then flag if extra code exists. For example, if a file named `jquery-3.3.1.min.js` is not an exact match to the real jQuery version (or has suspicious calls like `chrome.webRequest` inside it), that's an indicator the library was tampered with. In one report, malicious code was prepended to jQuery and another script in an extension to perform credential injection and proxy setup ³.

- **Alternate Execution Contexts:** Malicious extensions may abuse background scripts, content scripts, or even **DevTools pages** to hide behavior. Our analyzer should enumerate all extension entry points (background script/service worker, content scripts, optional scripts, etc.) and ensure we inspect each. Sometimes the malicious logic lives in a less obvious place (e.g., a hidden background script file or inside the `browser_action` popup script triggered only on clicking the extension icon). We should flag if the extension has an unused or seldom-used component with excessive permissions – e.g., a hidden options page that runs privileged code on load.
- **Update Mechanism Abuse:** Check the `update_url` in the manifest. Normally Chrome Web Store extensions use Google's update URL. If an extension (not enterprise) has a custom `update_url` pointing to an external site, it could be trying to bypass Web Store updates and pull a malicious update later (common in sideloaded or maliciously updated extensions). That's a risky indicator, though not always applicable if we're scanning a one-off CRX.

Heuristic: Create rules for **"likely evasion"**: - If code uses long timers or checks for certain dates/times before doing something sensitive, flag as possible logic bomb. - If the manifest or code hints at external update control (non-standard update URLs or fetches new code), increase risk. - Highly obfuscated code that only runs after a complex condition is also suspect – why would a benign extension hide a feature behind encrypted code unless it's trying to fool reviewers? ²⁶ .

By identifying these anti-analysis patterns, we not only catch sophisticated malware but also get context to avoid false positives (for example, if something is truly benign, it wouldn't need to hide functionality until after a delay).

6. Indicators of Compromise (IOC) Extraction and Correlation

As the analyzer scans the extension, it should extract IOCs that can later be cross-referenced or used in threat intelligence. Key IOCs include:

- **Domain Names and URLs:** Every external URL found (in strings or in fetch/XHR calls). E.g., `graphqlnetwork.pro`, `chatsaigpt.com`, etc. We list them in the output JSON (with context). The system can check these against known malicious domain feeds or previous scans. If a domain appears that was seen in another malicious extension, that strongly suggests a coordinated campaign.
- **IP Addresses:** Any hard-coded IP address in the code is suspicious (most legitimate services use domain names). IPs like `45.76.225[.]148` or similar, especially if they correspond to cloud/VPS providers, might indicate C2 servers ³⁴ . Extract those as IOCs.
- **File Hashes or Unique Strings:** If the extension contains a large blob (like an embedded binary or a large hex string), consider hashing it. Sometimes different extensions use the same packed payload; a matching hash can correlate them.
- **Extension IDs or Names:** In supply-chain scenarios, a campaign may target multiple extensions. If our analyzer has a database, it can note if the same suspicious code or domain is present in multiple extension IDs or if the extension's name closely mimics a known legitimate one (e.g., using "Google" or "ChatGPT" in the name improperly). That said, name-based detection is tricky and prone to FPs, but we can surface it for human analysts.

The analyzer's job regarding IOCs is to **surface them for further analysis or blocking**, not to decide maliciousness purely on that. But certain IOCs could themselves trigger a rule (e.g., domain matches a known phishing domain list -> high risk).

Reducing False Positives

While the above heuristics cast a wide net, not all extensions with “suspicious” traits are truly malicious. We need strategies to minimize false positives so the tool is practical for production use:

- **Multi-factor Scoring:** Instead of a binary label on a single rule, use a scoring model (e.g., a weighted sum or logistic regression) to combine signals. For instance, an extension using eval might be benign if it’s a small utility, but if it also has broad permissions and contacts unknown domains, the combined score would cross a threshold for malicious. Require multiple moderate-risk findings or one decisive high-risk finding to flag as malicious. This helps avoid flagging an extension just because it’s minified or just because it asks for one broad permission in isolation.
- **Whitelist Known Good Patterns:** Maintain allow-lists for common libraries and patterns:
 - If a script file exactly matches a known library (by hash or known filename pattern), we can ignore its content for malicious patterns (assuming CDN libraries aren’t modified). Many benign extensions bundle frameworks like Angular, React, etc., which appear as obfuscated code but are not malicious.
 - Known analytics or CDN domains: e.g., contacting `google-analytics.com` or a legitimate API like `api.dropbox.com` might be normal for an extension’s function. The IOC analysis could classify domains by reputation – well-known domains get less weight than random ones. However, attackers can also use cloud services, so still consider context (e.g., an extension whose core function doesn’t obviously need to talk to AWS but does so could be suspicious).
- **Contextual Permission Justification:** Try to infer the extension’s intent from its name or descriptions (if available in the package or from store metadata). For example, an extension called “Cookie Editor” will request `cookies` and `<all_urls>` legitimately. But an extension called “Theme for Gmail” should not need `tabs` and `webRequest`. If we have the description, a simple NLP or keyword check might align permissions with claimed features and reduce false flags on security tools or developer tools that genuinely need broad access.
- **Severity Tiers:** Each rule can produce a severity level (low/med/high). We can choose to only auto-flag extensions that hit at least one high-severity indicator or several medium ones. For instance, detecting actual password grabbing code or known malicious C2 domain could be a **High** severity hit (almost certainly malicious) whereas “minified code” alone is **Low** (common in benign extensions). The tool might output all findings but only mark the extension as “*Malicious: Yes*” if the combined confidence is above a threshold. Google’s internal system aimed for near-zero false positives, even if that meant some malware slips through, because false removal of an extension can impact many users ³⁵. We can emulate that caution by tuning thresholds and perhaps requiring human review for borderline cases.
- **User Opt-in Warnings:** For certain moderate-risk extensions, the tool could label them “Suspicious” rather than outright “Malicious”. This might be useful if integrating into a browser or enterprise system – suspicious extensions can be flagged for review or monitored closely, reducing the chance of blocking a benign one incorrectly.
- **Continuous Updates and Learning:** Attackers adapt, so our heuristics should too. Keep an eye on new malicious techniques reported in the wild and update rules accordingly. A feedback loop (analyst marking a false positive or confirmed missed malware) can help recalibrate the scoring model. For example, if a false positive pattern is found (say many benign dev extensions trigger a certain rule), we adjust that rule’s weight or add an exception condition.

By combining these practices, we ensure the analyzer remains accurate and “*production-ready*”, focusing analyst attention on truly likely threats and not flooding with noise.

Prioritized Backlog of Detection Features

Implement the static analyzer iteratively, starting with the most impactful heuristics. Below is a **prioritized backlog** of features/rules to build, roughly in descending priority:

1. **Permission Risk Scoring (High Priority)** – Parse `manifest.json` for permissions and host access. Implement the risk score model for permissions (critical combos like `<all_urls>` + `webRequest` + `cookies` etc. get highest points). This provides an initial risk baseline ² ¹¹.
2. **Domain & URL IOC Extraction (High Priority)** – Scan all files for domain names and URLs. Flag unknown or low-reputation domains (could integrate with a reputation list). This directly catches known malicious infrastructure ⁹ ³⁶.
3. **Sensitive API Usage Rules (High Priority)** – Static patterns for cookie access, credential access, keylogging, form grabbing, and network interception:
4. e.g. “uses `chrome.cookies` on `<all_urls>`” or “content script reads `document.cookie`” – likely session theft.
5. “Registers `chrome.webRequest` listener” – inspect what it does (if reading headers, likely data theft).
6. “Adds keypress/submit listeners on pages” – potential keylogger.
7. “Scrapes specific sites or DOM elements (e.g., looking for `.chatGPT` elements)” – targeted data theft ⁶.
8. **Obfuscation Detector (High Priority)** – Implement code entropy checks and known packer signatures ²⁷. At minimum, flag any file with highly obfuscated content (this often correlates with malicious intent, or at least warrants closer review).
9. **Dynamic Code Execution Check (Medium Priority)** – Identify any eval or dynamic script injection. This includes scanning for `eval()`, `new Function()`, template literals that look like code, or DOM insertion of scripts. Also flag usage of `chrome.scripting.executeScript` or suspicious `postMessage` / `executeScript` patterns.
10. **Remote Config/Payload Fetch Detection (Medium Priority)** – Flag any network call that appears to retrieve config or code. For instance, `fetch('http://...')` followed by logic storing or parsing the result. If possible, simulate simple constant propagation to see if that result is used in an eval or injected context.
11. **Hidden or Dead Code Detection (Medium Priority)** – Analyze the extension’s files to see if any script is not obviously triggered or if permissions are unused. This could involve scanning for event listeners or entry points. Unused but sensitive code could indicate dormant malware.
12. **Anti-Analysis Patterns (Lower Priority)** – Implement specific checks for delayed execution (e.g., `setTimeout` with very large values, usage of `Date()` or timers in suspicious context) and environment checks (like reading navigator properties). These are lower priority but good for catching advanced malware.
13. **Library Fingerprinting & Whitelisting (Lower Priority)** – Build a small library of known file hashes for common libraries to avoid flagging them. This will reduce noise from, say, react or lodash minified files. Also, flag if those files have unexpected modifications (size mismatch or extra code).
14. **Machine Learning Classifier (Future Enhancement)** – Once enough data is collected, train a model to classify extensions using the extracted features (permission score, API patterns, obfuscation score, etc.). This can complement the rules with an additional layer, potentially catching subtle patterns. However, it should be used with explainability (so we can present which features led to the decision) to maintain analyst trust ³⁷.

15. **Dynamic Analysis Integration (Future)** – Though our focus is static, consider a sandboxed execution for high-risk cases (e.g., run the extension in a controlled browser environment to see if it actually exfiltrates data). Projects like Hulk and JavaSith have shown dynamic analysis can reveal behaviors that static might miss ³⁸ ³⁹. This is complex to implement but could be a later phase for a more comprehensive tool.

Each implemented feature should be tested against known malicious and benign extensions to fine-tune its accuracy. Starting with manifest and obvious code signals (top of the list) will catch a large percentage of common threats with relatively low effort.

Suggested JSON Output Schema

When the analyzer scans an extension, it should output a clear report of findings in JSON. The schema should include **rule hits with descriptions, IOCs, and confidence scores** to facilitate automated ingestion or analyst review. Below is a suggested schema with an example structure:

```
{
  "extensionId": "abcdefghijklmno",
  "extensionName": "Example Extension",
  "riskScore": 85,
  "riskLevel": "High",
  "findings": [
    {
      "ruleId": "PERM001",
      "rule": "Broad Host Permissions with Network Access",
      "description": "Extension has host_permissions '*/**/*' and uses
webRequest blocking.",
      "severity": "High",
      "confidence": 0.9
    },
    {
      "ruleId": "CODE003",
      "rule": "Suspicious Cookies Access",
      "description": "Content script reads document.cookie on all pages.",
      "severity": "High",
      "confidence": 0.95
    },
    {
      "ruleId": "NET002",
      "rule": "External Data Exfiltration",
      "description": "Found fetch calls to external domain
'malicious.example.com'.",
      "severity": "High",
      "confidence": 0.9
    },
    {
      "ruleId": "OBF001",
```

```

    "rule": "Obfuscated Code Detected",
    "description": "background.js appears heavily minified/obfuscated (eval
and encoded strings present).",
    "severity": "Medium",
    "confidence": 0.8
  }
],
"indicators": [
  {
    "type": "domain",
    "value": "malicious.example.com",
    "context": "Found in background.js:55 (exfiltration target)"
  },
  {
    "type": "url",
    "value": "https://attacker.io/config.json",
    "context": "Found in manifest 'update_url' field"
  }
]
}

```

Explanation: The top-level includes an overall `riskScore` (e.g., 0-100 or a percentile) and a `riskLevel` categorization (Low/Medium/High) for quick reference. Under `findings` (which could also be called `ruleHits`), each object represents a triggered rule or heuristic: - `ruleId` is an internal identifier for the rule. - `rule` is a short name. - `description` provides details (what was detected and why it's risky). - `severity` indicates the rule's importance. - `confidence` is how sure we are this indicates malicious behavior (could be derived from rule weight or model probability).

The `indicators` list compiles IOCs like domains or URLs. Each IOC entry can include a `context` (e.g., which file/line or usage made us collect it). Additional IOC types might include `ip`, `fileHash`, etc., if relevant.

This JSON format cleanly separates **what was found** from the extension and the **assessment of risk**, making it easy for a consuming system to take action (like block the extension, alert an analyst, or aggregate stats). By providing the rule descriptions and IOCs, we ensure a human can understand why an extension was flagged – crucial for verifying true vs false positives.

Sources: The heuristics and examples above are drawn from recent research on malicious extensions and real-world incidents. For instance, supply-chain attacks have injected background scripts that fetch configs to steal tokens (e.g., targeting ChatGPT and Facebook Business accounts) ¹ ²⁰. Advanced malware like *Download Manager Integration* used obfuscated payloads and clever DOM tricks to execute remote code on every page ²³ ²². Other extensions have intercepted web traffic to hijack sessions or credentials (e.g., the *Phantom* proxy extensions) ⁷ ⁴. Fake AI extensions illustrate how attackers scrape chatbot data and user content under the guise of AI features ⁵ ⁶. By encoding these patterns into static rules and a scoring model, our analyzer can detect many of these threats. Combining permission analysis, code pattern

matching, and IOC correlation provides a robust static approach, which, when tuned properly, achieves high accuracy in identifying malicious Chrome extensions while keeping false positives low 35 2 .

1 9 20 28 29 34 Targeted supply chain attack against Chrome browser extensions - Sekoia.io Blog
<https://blog.sekoia.io/targeted-supply-chain-attack-against-chrome-browser-extensions/>

2 11 15 Behind the Mask of Madgicx Plus: A Chrome Extension Campaign Targeting Meta Advertisers
<https://www.cybereason.com/blog/chrome-extension-campaign-madgicx>

3 4 7 13 14 18 25 Two Chrome Extensions Caught Secretly Stealing Credentials from Over 170 Sites
<https://thehackernews.com/2025/12/two-chrome-extensions-caught-secretly.html>

5 6 36 Two Chrome Extensions Caught Stealing ChatGPT and DeepSeek Chats from 900,000 Users
<https://thehackernews.com/2026/01/two-chrome-extensions-caught-stealing.html>

8 12 39 JavaSith: A Client-Side Framework for Analyzing Potentially Malicious Extensions in Browsers, VS Code, and NPM Packages
<https://arxiv.org/html/2505.21263v1>

10 19 21 22 23 24 26 32 33 Analysis of an advanced malicious Chrome extension | Almost Secure
<https://palant.info/2025/02/03/analysis-of-an-advanced-malicious-chrome-extension/>

16 17 31 Prevent Breaches by Spotting Malicious Browser Extensions
<https://www.crowdstrike.com/en-us/blog/prevent-breaches-by-stopping-malicious-browser-extensions/>

27 30 35 37 research.google.com
<https://research.google.com/pubs/archive/43824.pdf>

38 pergamos.lib.uoa.gr
<https://pergamos.lib.uoa.gr/uoai/object/2795448/file.pdf>